

MQTTSec: An Intelligent Framework to Secure the MQTT Protocol Against Flooding-based DoS Attacks

Debasmita Dey, Nirnay Ghosh*

*Department of Computer Science and Technology, Indian Institute of Engineering
Science and Technology, Shibpur, Howrah 711103, India*

Abstract

Message Queuing Telemetry Transport (MQTT) is a widely used application layer protocol in the paradigm of Internet of Things (IoT). It follows a publish-subscribe model where a central entity, the MQTT broker, routes messages from publishers to subscribers based on topics. MQTT also ensures reliability, Quality of Service (QoS), and topic-based communication, which makes it suitable for resource-constrained scenarios. However, it is vulnerable to Denial of Service (DoS) attacks such as basic CONNECT flooding, fast flooding, and heavy flooding. These attacks overwhelm the broker with a high rate of CONNECT requests without completing the session or sending valid data, resulting in resource exhaustion, delay in legitimate client-server interactions, and network service disruptions. In this work, we propose a Reinforcement Learning (RL) based framework, *MQTTSec*, that enables the MQTT broker to learn optimal actions using state-policy mappings dynamically. The state representation is derived from a machine learning classifier that detects whether MQTT traffic is benign or malicious, enabling the broker to make intelligent decisions, such as accepting or declining requests and issuing a warning signal to potentially malicious publishers. As suitable datasets for training and testing such models are not readily available, we curated synthetic ones using a Latent Diffusion Model (LDM). For validation, we create a simulation environment consisting of one MQTT broker and two MQTT publishers. Finally, we present the results related to dataset generation, classification of MQTT traffic, and RL-based learning to evaluate the broker's performance, which establishes that *MQTTSec* can efficiently detect and prevent DoS attacks in a time-sensitive MQTT environment.

*Corresponding author

Email address: nirnay@cs.iiests.ac.in (Nirnay Ghosh)

Keywords: MQTT, DoS attack, Latent Diffusion Model, Reinforcement Learning

1. Introduction

With each passing year, the use of IoT-enabled devices in different applications like smart home, smart healthcare, etc., is increasing at a rapid pace. These devices have different makes, models, and use heterogeneous platforms to generate, process, and share the sensory data they collect from the environment. Managing such a degree of heterogeneity due to interoperability issues is a hard task. Message Queuing Telemetry Transport (MQTT) protocol stands out to be one of the few protocols that support interoperability [1] and helps to address this issue, allowing heterogeneous IoT devices to share messages through a central entity, called the “broker.”

MQTT is the de facto application layer protocol in the IoT protocol stack. The IoT application layer typically handles the task of transferring messages between devices. MQTT is a lightweight, publish-subscribe messaging protocol designed for efficient communication in resource-constrained and low-bandwidth environments such as IoT networks. It operates on a client-server model where the MQTT broker acts as the central server managing all message traffic between clients. Clients can either publish messages to specific topics (publisher) or subscribe to receive messages on those topics (subscriber).

Although the protocol ensures secure message exchanges in terms of confidentiality and integrity, problems regarding the availability of the MQTT service are prevalent. These problems may be due to unchecked replay attacks, man-in-the-middle attacks, and Denial of Service (DoS) attacks. Among these, the stealthiest is the DoS attack [2], which overwhelms the network and exhausts its resources by performing the same actions repeatedly. Moreover, this kind of attack not only affects the victim entity, but due to the unavailability of services, other connected components (clients) also experience delays, disruption of services, and eventually termination of the connection. A plethora of works [3], [4], [5] have been proposed for early detection of DoS-based attacks on the MQTT protocol. Most of these works incorporate machine learning approaches on MQTT-based datasets to train a classifier and then use it for distinguishing traffic in real-time. No prominent work has been reported that prevents a DoS-like scenario in an MQTT-enabled IoT network. Moreover, the state-of-the-art detection mechanism closes the connection completely without providing any scope

for correction, as sometimes there may be false positives, where the client may not be malicious, but network conditions like the absence of a rate-limiting mechanism, poor load balancing, and low bandwidth availability also lead to a DoS-like situation.

In this work, we devise a framework, *MQTTSec*, for making intelligent decisions on the “basic CONNECT flooding”, “fast flooding”, and “heavy flooding” DoS attack [6] launched against the MQTT broker by integrating reinforcement learning (RL) and machine learning (ML) models. Unlike supervised and unsupervised machine learning, RL depends on the feedback received in terms of rewards and is free from data-based training, making it suitable for real-time decision-making. To leverage this advantage, we use RL in developing a robust and intelligent broker for the MQTT that thwarts DoS-based attacks. *MQTTSec* enables the broker to go beyond the conventional connection termination as the sole preventive measure by issuing warning signals for misbehaving publishing clients before suspending them from the network.

In a publish-subscribe model like MQTT, message exchanges among entities such as publishers, subscribers, and one or more brokers take place in real time. Such a real-time MQTT traffic dataset is not readily available in the public domain. In its absence, the downstream ML model will never be optimally trained. Therefore, to address this limitation, we generate synthetic data samples using a Latent Diffusion Model (LDM) [7]. Below, we discuss a use case from a smart home environment that uses MQTT to motivate this work.

Motivating Example: In a smart home environment, several IoT de-

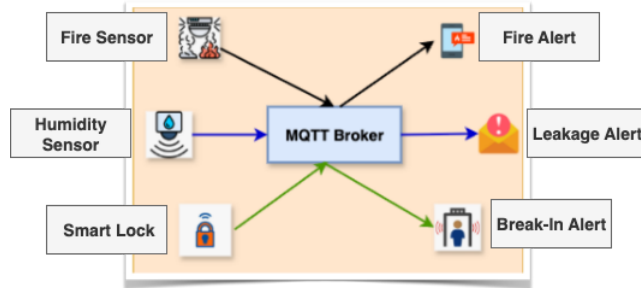


Figure 1: Motivating Example

vices are connected to a gateway device to automate and control household functions like lighting, temperature, and security (see Fig. 1). The gateway device may host the MQTT broker services. MQTT protocol enables efficient communication between these devices by providing a lightweight

publish-subscribe messaging model. The MQTT broker receives messages from sensors embedded in various devices (acting as publishers), such as fire alarm systems, smart humidifiers, smart locks, and publishes them to the concerned applications that have subscribed to different topics offered through the MQTT messaging service. It is useful in emergency scenarios, such as a fire breakout or a theft attempt, when the publisher-broker-subscriber entities operate in coordination to generate an alert for the smart home owner. But in a situation where any IoT device is compromised by an adversary, it can flood the broker with messages, resulting in a DoS-like scenario. As a precautionary measure, the broker will suspend all existing connections. This will disrupt the coordination among the entities and may lead to safety and security problems in the home environment. Thus, there is a need for an intelligent solution that prevents the broker from abruptly suspending all connections, enables it to issue a warning signal before disconnecting, and selectively closes only the malicious/faulty ones.

In this work, we consider three flooding attacks: basic CONNECT flooding, fast flooding, and heavy flooding. The ML model carries out MQTT traffic classification and provides the state as “attack” or “benign” to the RL. The RL model, in turn, takes actions and provides feedback to the environment. Below, we highlight the major contributions of this work.

- We design and implement a framework, *MQTTSec*, using supervised and reinforcement learning, to enable an MQTT broker to detect three flooding-based DoS attacks and prevent them using the message features.
- We generate a synthetic dataset of “benign” and “attack” MQTT traffic using the Latent Diffusion Model and existing public datasets.
- We implement a simulated MQTT-enabled network and carry out performance analysis through the generation of a curated dataset, classification using an optimal ML model, and validation using the RL model.

The rest of the paper is organized as follows. Sec. 2 reviews the related works on DoS attacks in MQTT, their detection using ML, and the generation of synthetic network-related data. We discuss the background concepts for MQTT, Latent Diffusion model, and RL in Sec. 3. Sec. 4 presents the system and threat models. In Sec. 5, we discuss the proposed *MQTTSec* framework, which includes the ML classifier, the synthetic data generation using LDM, and the RL model for taking intelligent decisions against the

DoS. The results related to the LDM, ML, and RL models are presented in Sec. 6. Finally, Sec. 7 concludes the paper with future work directions.

2. Related Works

This section reviews the severity of DoS attacks in an MQTT-enabled network and state-of-the-art approaches for its detection and prevention.

2.1. DoS attack in MQTT

The upsurge of IoT devices has culminated in the use of the MQTT protocol for facilitating the ease of communication. However, the security issues always remain a concern; more evidently, the DoS attacks cause much damage to the services provided by the network. The authors in [8] constructively describe two categories of DoS attacks in MQTT, one that saturates the resources of the broker and another that saturates the resources of the client. Slow DoS attack using the *KeepAlive* mechanism of MQTT is proposed in the work [9]. The authors in [10] suggest the effects of DoS attacks in terms of resource exhaustion and unavailability of services. In [11], the authors discuss the effects of the DoS attack on the various message types of MQTT.

2.2. Detection of DoS Attacks

Multiple works exist for the detection of DoS attacks in MQTT using machine learning approaches; one such approach is [4], where detection is performed using an *AODE* classifier, a decision tree, and a multilayer perceptron. Through experiments, the authors conclude that the accuracy of the classifiers increases when packet size and field length-based features are also considered for classification. In [3], the authors use fuzzy logic-based systems for detecting malicious behavior of MQTT-based nodes. However, fuzzy-based detection depends solely on a set of static rules, making it non-flexible and susceptible to single-point failure. The *AdaBoost* classifier with default parameter settings has been utilized by [12] for DoS attack detection in IoT environments. *AdaBoost* is sensitive to noisy and unbalanced data, resulting in generating incorrect results. The authors in [13] propose DoS attack detection using three types of classifiers: multilayer perceptron, random forest, and support vector machine. The difficulty with these algorithms lies in their inability to work with real-time data.

Limitations in the State-of-the-art: In the state-of-the-art, there is a dearth of work on the prevention of DoS attacks in MQTT. Moreover, it can be observed that very few works have focused on working in real-time;

therefore, in IoT environments where applications are inherently real-time, a considerable inadequacy exists as far as real-time detection and prevention of DoS attacks in MQTT-enabled networks. In this work, we use an optimal ML classifier for classifying normal traffic from the attack traffic in MQTT. For the prevention, we leverage reinforcement learning, which enables the MQTT broker to learn over time about taking the optimal action on-the-fly.

3. Background

In this section, we provide a brief overview of the background concepts essential for this work.

3.1. Message Queuing Telemetry Transport (MQTT) Protocol

MQTT protocol follows the topic-based publish/subscribe model for message sharing. It has three main components, namely the broker, publisher, and subscriber. Below, we provide a brief overview of MQTT.

1. Topic: MQTT topics ¹ act as the addressing system for messages, enabling devices and applications to structure and control how information is routed and filtered. They follow a hierarchical format ² that can represent different elements of an IoT system, such as device locations, sensor categories, or data types. In MQTT’s publish/subscribe architecture, publishers send messages on designated topics, and subscribers register for those topics to receive relevant messages. The MQTT broker manages this exchange by receiving messages from publishers and distributing them to all clients subscribed to the corresponding topics, ensuring efficient and organized message delivery.

2. Broker: The broker is the central entity responsible for managing all message routing between publishers and subscribers. It acts as an intermediary, receiving messages from publishers and forwarding them to all clients subscribed to the relevant topic(s). This decoupled communication model allows publishers and subscribers to remain unaware of each other’s presence, thereby improving scalability, flexibility, and robustness in dynamic networks [14]. The broker also enforces *Quality of Service (QoS)* guarantees by storing and retransmitting messages as needed, depending on the level (QoS 0, 1, or 2) chosen by the publisher. In addition to routing, the broker may handle authentication, authorization, and access control, ensuring

¹<https://docs.aws.amazon.com/iot/latest/developerguide/topics.html>

²<https://docs.solace.com/API/MQTT/MQTT-Topics.htm>

that only legitimate clients can publish to or subscribe to certain topics [15]. MQTT brokers such as Mosquitto ³, HiveMQ ⁴, and EMQX ⁵ are widely used in smart industries due to their lightweight implementation and support for high-throughput, low-latency messaging.

3. Publisher: Publisher is a client that initiates message exchanges on specific topics with the MQTT broker. It serves as the primary source of data in the publish-subscribe model, commonly used in IoT systems, telemetry, and messaging infrastructures. A publisher does not need to know the identity or number of subscribers; instead, it simply publishes messages under a topic name, and the broker handles message delivery to all clients subscribed to that topic. This loose coupling between message producers and consumers enhances system scalability and flexibility [14]. Publishers can control the reliability of message delivery by selecting an appropriate QoS level. MQTT supports three QoS levels that differ in the number of acknowledgments (ACK) received: (i) QoS 0 (no ACK), QoS 1 (one ACK for all messages exchanged in a session), or QoS 2 (one ACK per message exchanged) [16]. Fig. 2 illustrates the message exchanges among publisher, broker, and subscriber at various QoS levels.

4. Subscriber: In MQTT, the subscriber is a client that expresses interest in receiving messages associated with specific topics. Unlike traditional request-response models, the subscriber in MQTT remains passive; instead, it relies on the broker to deliver messages that match its subscription filters. When a publisher sends a message on a topic, the broker routes that message to all subscribers of that topic, enforcing the QoS level defined by either the publisher or the subscriber (whichever is lower). Subscribers can also specify session persistence, by which the broker shall queue messages sent while the former is offline and deliver them when it reconnects, depending on the session state [15]. Additionally, MQTT offers a caching service, where a subscriber will immediately receive the last retained message on a topic upon subscribing. These features make subscribers highly adaptable and efficient for use cases ranging from simple telemetry dashboards to critical alert systems in IoT applications.

In the proposed *MQTTSec* framework, we are not considering the operations of the subscriber, as the attacks we attempt to mitigate (discussed in Sec. 4) originate from the publishers' side.

³<https://mosquitto.org/>

⁴<https://www.hivemq.com/>

⁵<https://www.emqx.com/en>

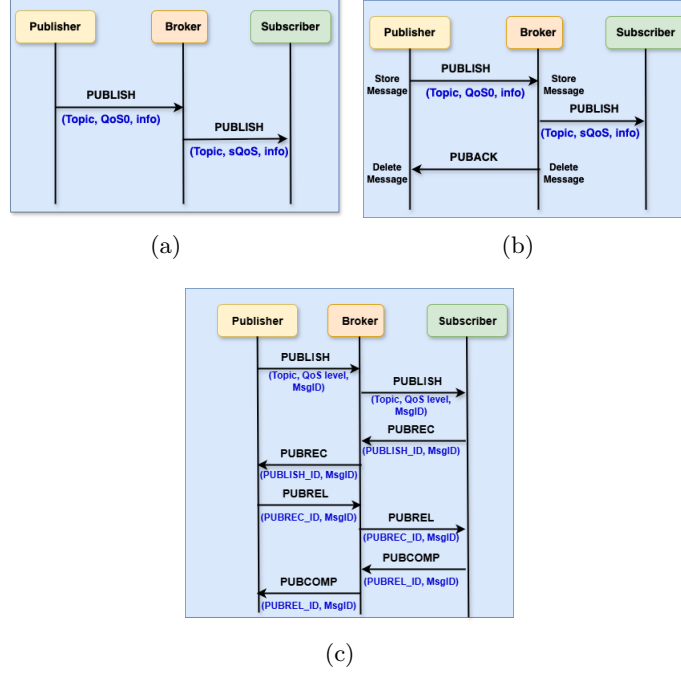


Figure 2: MQTT Message Exchanges following Connections Establishment: (a) QoS 0: The publisher first sends the PUBLISH message containing information about the topic and QoS level to the broker. The broker, upon receiving the message, relays the same to the concerned subscriber who has subscribed to the particular topic. (b) QoS 1: After receiving the PUBLISH message from the broker, the subscriber sends the PUBACK message to the publisher. (c) QoS 2: Here, the publisher sends a PUBLISH message, and the broker responds with a PUBREC message. The publisher then replies with PUBREL, and the broker completes the exchange with PUBCOMP, ensuring the message is delivered exactly once with no duplication.

3.2. Latent Diffusion Model (LDM)

Well-curated network datasets are largely unavailable in the public domain. This has prompted researchers to leverage generative AI and other mechanisms to generate synthetic samples. GAN (Generative Adversarial Network) has been used in [17] and [18], where the authors generate IP and QUIC network packets from noise. However, GAN is hard to train due to oscillation of the model parameters, resulting in destabilization of the network and hence delay in convergence [19]. In [20], the authors generate synthetic packets using OpenAI’s Generative Pre-trained Transformer 3 (GPT-3). Due to its high computational and memory requirements, GPT-3 is not practical for integration into MQTT-based IoT systems with limited

resources [21]. Deep reinforcement learning has been used in [22] to generate adversarial packets by taking raw malicious packets as input. The works in [23], [24], and [21] employ diffusion models operating in the pixel domain to generate network packets across various layers of the protocol. Higher-dimensional pixel space may be computationally expensive for constrained IoT environments, which led us to adopt the latent diffusion model (LDM).

The concept of the diffusion model is adding noise to the original data, and gradually, the noisy data is denoised to retrieve the original data (refer to Fig. 3). The process of adding small amount of Gaussian noise to a clean sample X_0 is termed as forward diffusion, whereas backward diffusion (denoising) learns the reverse transitions and successively removes noise to map X_T back to X_0 .

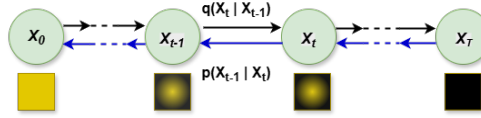


Figure 3: Diffusion Process

As shown in Fig. 3, a sequence of noisy latent variables X_0 to X_T is generated using the Markov process, and at each time step t , a small amount of Gaussian noise is added using Eqn. 1, where X_t is the noisy latent at t and X_{t-1} is the latent from the previous step. β_t is the small noise parameter for controlling the amount of noise to be added at time t . According to the definition of the Gaussian distribution [25], $\sqrt{1 - \beta_t}X_{t-1}$ is the mean and $\beta_t I$ is the variance, with I being the identity matrix.

$$q(X_t|X_{t-1}) = \mathcal{N}(X_t; \sqrt{1 - \beta_t}X_{t-1}, \beta_t I) \quad (1)$$

The backward diffusion process is given by Eqn. 2. Here, $\mu_\theta(X_t, t)$ is the predicted mean by the backward diffusion model neural network, $\Sigma_\theta(X_t, t)$ is the predicted variance, and θ is the parameter of the neural network trained for denoising.

$$p(X_{t-1}|X_t) = \mathcal{N}(X_{t-1}; \mu_\theta(X_t, t), \Sigma_\theta(X_t, t)) \quad (2)$$

Generally, these models need high-dimensional data spaces; however, the LDM [7] also works with a low-dimensional latent space. The latent space is formed using autoencoders, which compress the high-dimensional input data into a low-dimensional compressed space. The overall idea is to add noise to this latent space through forward diffusion, and then predict the noise added at each step using backward diffusion. The backward diffusion

process minimizes the loss calculated using the mean squared error loss in Eqn. 3, which measures the difference between the actual noise ξ and the predicted noise $\xi_0(X_t, t)$ across different timesteps t , where X_t is the noisy latent at diffusion step t .

$$L = E_{X_0, \xi, t} [\|\xi - \xi_0(X_t, t)\|_2^2] \quad (3)$$

In the original design [7] of the LDM, the authors also use the conditioning mechanism, which helps with the conditional synthesis of images based on a variety of texts. In our case, we are generating CSV data from an existing CSV file, and do not require the conditioning mechanism. We use LDM to generate synthetic datasets for *MQTTSec* (details in Sec. 5.1).

3.3. Reinforcement Learning

Reinforcement Learning (RL) [26] is a branch of AI where an agent learns to make decisions by interacting with an environment to maximize cumulative rewards over time. Unlike supervised ML, where the model learns from labeled data, RL agents learn through trial-and-error, guided by a reward function. RL is modeled as a Markov Decision Process (MDP) where the agent observes a state, takes an action, receives feedback in the form of a reward and a new state, and continues as long as it is rewarded (or penalized).

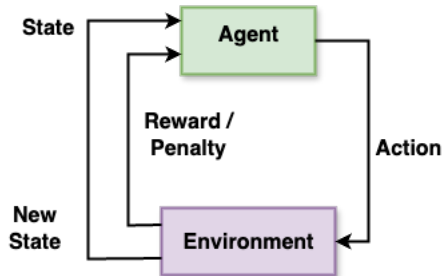


Figure 4: Reinforcement Learning

Fig. 4 provides a basic block diagram of the various components of RL, where it is evident that the action taken by the agent on the environment results in a reward/penalty and a new state, which in turn influences it to take the next action.

The goal of RL is to learn an optimal policy, a mapping of states to actions, that maximizes the expected return. The quality of a particular state after taking an action can be determined using Q -learning, policy gradient, and Monte Carlo mechanisms.

The Q -learning helps the agent make the best decision by interacting with the environment. The agents gather experience from the environment by taking various actions and store these experiences in the form of values in a tabular format known as a Q -table. The Q -values in the table are updated as [26]:

$$Q(s, a) = R(s, a) + \gamma \max_a Q(s', a) \quad (4)$$

Here, $Q(s, a)$ is the Q -value for given state-action pair, $R(s, a)$ is the reward for taking action a in state s , γ is the discount factor, and $\max_a Q(s', a)$ is the maximum Q -value for the next state s' and action a .

Gradient-based methods in RL, known as policy gradient methods, directly optimize the parameters of the policy by estimating the expected return gradient concerning those parameters [26]. These methods are particularly useful for continuous or high-dimensional action spaces where value-based methods (like Q -learning) may struggle.

The Monte Carlo mechanism is a method where the value of a state or an action is estimated purely from the actual returns observed after complete episodes of interaction, without using any bootstrapping from future estimates (as in Q -learning). The agent runs full episodes, collects the sequence of rewards, and computes the return G_t for each state or state-action pair visited. This return is then averaged over many visits to approximate the expected value. Monte Carlo methods are particularly suited for episodic tasks where the outcome is only fully known at the end of an episode. For *MQTTSec*, we are using the Monte Carlo mechanism.

4. Threat and System Models

This section presents the threat and system models of the proposed *MQTTSec* framework.

4.1. Threat Model

This section examines the potential threats that MQTT may face during the connection establishment, QoS 0 message exchange, and QoS 1 message exchange phases, which the proposed framework attempts to mitigate. As mentioned, MQTT is susceptible to flooding DoS attacks, which can saturate a MQTT broker's resources, reduce service availability, and make the network non-functional. *MQTTSec* detects and prevents three types of flooding DoS attacks that are as follows:

1. Basic CONNECT Flooding in Connection Establishment: This scenario occurs when the client attempts to authorize itself to the server at the time

of connection establishment [27] (refer to Sec. 3.1). In the process, the client sends a series of *CONNECT* messages until the server responds with a *CONNACK* message.

2. Fast Flooding in QoS 0: In this scenario, the attack occurs during the QoS 0 message delivery, when the publisher continues to send *PUBLISH* packets at an increasingly high rate [28]. This condition leads to the overwhelming of the broker, and hence, the subsequent termination of the connection.

3. Heavy Flooding in QoS 1: This scenario occurs when the publisher and/or subscriber repeatedly sends *PUBLISH* messages [12] with a larger payload size at a higher rate until the broker sends a *PUBACK* message, leading to the saturation of the broker’s resources.

In case of a flooding DoS attack, the attacker generates a large volume of traffic towards the MQTT broker to overwhelm it and exhaust its resources. This condition affects all other components of the MQTT ecosystem, leading to a complete shutdown, and hence violates the “availability” requirement of the CIA security triad. Eventually, the system’s reliability and trust are also compromised, leading to operational and economic losses.

The proposed *MQTTSec* framework attempts to detect flooding DoS attacks and prevent the complete shutdown of the network using classification of the MQTT traffic and RL-based decision making.

4.2. System Model

Our system model uses the standard model of the MQTT protocol integrated with a supervised learning-based traffic classification model and an RL-based decision system. We generate synthetic data for the validation of the ML and RL modules by using an LDM.

We consider two types of entities, namely the client (benign and malicious) and the server, in the system model. Here, the client refers to the publisher or subscriber, and the server is the broker. Besides the two levels of QoS message exchange, we are also taking into account the DoS attack during the TCP connection establishment phase.

It is evident from the above-discussed threat model that, out of the three types of attacks, to identify the “BASIC connect flooding” and “heavy flooding,” we require indicators related to the length of the message and time. On the other hand, the “fast flooding attack” requires a time-dependent approach. Therefore, we consider two important parameters for policy formation of the RL model of *MQTTSec*:

(i) Length of the message: it refers to the length of the payload size, and as

Action (Λ): There are two categories of actions, that is, episode-level and epoch-level. The episode-level actions include “accept”, “decline”, and “warn.” The epoch-level actions are “suspend” or “continue” with the client based on the return and ϵ value.

Reward (R): We denote the rewards for episode-level actions as +2 for “warning”, +1 for “accept”, and 0 for “decline” actions. Our objective is to prevent the DoS attack in the MQTT network, and avoid the complete shutdown of the same; therefore, we have assigned the highest reward for the “warning” action.

Return (G): We calculate the discounted return of each client (publisher) at the end of the epoch, based on this return value, the agent takes another set of actions.

Policy (P): There are two kinds of policies in the RL model, (i) episode level policy: here the agent uses the parameters like message length and time delta to accept or decline the message or warn the client; (ii) epoch level policy: here the agent uses the rewards obtained by the clients in each episode to stochastically take actions against them at the end of an epoch.

ϵ -Greedy: The RL agent applies the ϵ -greedy approach at the end of an epoch on the discounted return obtained by the clients to stochastically choose between ‘exploration’ and ‘exploitation.’

The RL model also maintains a reward buffer, which stores the reward obtained by each client in each episode. The agent fetches the rewards for each client at the end of an epoch to calculate the return. The states are determined from the return values, that is “high return” or “low return,” and the actions are “continue” or “suspend” the client. The client, on receiving the action (warning, decline, or accept) from the broker, may respond with a rectified behavior of sending messages at a reasonable rate.

We use a supervised ML model, trained on publicly available and synthetically generated datasets. The class labels are (i) attack traffic: class label 1; and (ii) benign traffic: class label 0. The ML model is initially trained offline, i.e., before any connection establishment or message passing. Then, at the start of an epoch, in every episode, when the MQTT broker receives a request or message (traffic) from the publisher, the pre-trained ML model classifies these packets either as attack traffic or benign traffic. The RL model, in turn, will use this classification result as its state and take actions and provide rewards accordingly (refer to Sec. 5.3). The MQTT broker stores these packets in its local database and uses them to retrain the ML model. The LDM generates synthetic datasets that we use for training and retraining the ML model. Table 1 presents the meanings of the symbols used in this work.

Table 1: Symbol Table

Symbols	Meaning
E	Number of epochs for retraining ML model
ϵ_c	Residual ϵ
Υ_n	Classification error of the n^{th} episode of the last epoch
A_E	Action at the end of an epoch
τ_E	Threshold for Return
R_i	Reward for i^{th} client in RL
θ_{2avg}	Average threshold for time delta
γ	Discount factor
θ_{1min}	Minimum threshold for length of the message
θ_{1max}	Maximum threshold for length of the message.
θ_{2min}	Minimum threshold for time delta
θ_{2max}	Maximum threshold for time delta
E	Environment in RL
S	States in RL
A	Agent in RL
Γ	weight given to accept action
α	Tuning parameter for θ_2
β	Tuning parameter for θ_1
P	Policy in RL
Λ	Action in RL
τ_E	Threshold for the Return value at the end of an epoch
Υ_n	Accuracy of the n^{th} episode
n	Number of episodes in an epoch
Δ	Suspension duration for the client

5. MQTTSec Framework

In this section, we present a detailed layout of the proposed *MQTTSec* framework, which attempts to protect the MQTT-enabled network against the “basic CONNECT flooding,” “fast flooding,” and “heavy flooding” DoS attacks. The following three subsections elucidate three main components: synthetic packet generation, ML model, and RL model.

5.1. Latent Diffusion Model for Dataset Generation

The objective of the proposed *MQTTSec* framework is to enable an RL agent (MQTT broker) to learn continuously about the behavior of clients

based on classified incoming MQTT traffic. To facilitate such a learning ecosystem, there is a need for a continuous flow of MQTT traffic, which consists of both attack and benign data for the ML model to classify. The classification result will be used by the RL agent to make decisions. Since such a tailor-made dataset for training is not publicly available, we synthetically generate MQTT traffic using the latent diffusion model (LDM). We train the LDM using the publicly available datasets to generate both classes of synthetic data.

Fig. 6 provides an overview of the LDM architecture, which is used for generating synthetic datasets. As shown, we first create a low-dimensional latent space from the raw data (given as CSV values) using an encoder. The encoder is a feedforward sequential neural network, which contains a series of linear layers and rectified linear unit (ReLU) activation functions between them. In the next step, we perform the forward diffusion process, which is basically a Markovian process of transforming a structured data x_0 to a noise-like distribution x_T . The “noisy latent space” x_T goes through the backward diffusion process to predict the added noise. In the backward diffusion process, we use the customized *UNet1D* architecture [29] to learn the denoising function that maps noisy latent representations towards cleaner versions at each timestep.

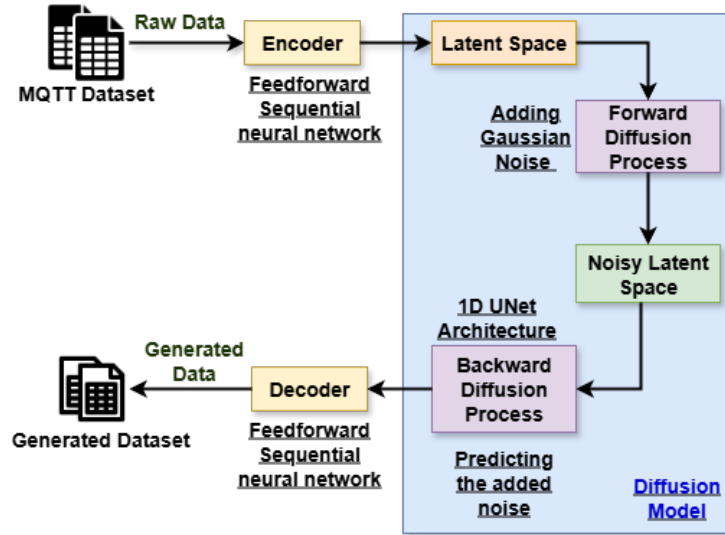


Figure 6: LDM Architecture

The *UNet1D* architecture used here is specifically tailored for 1-D sequential data [29]. As shown in Fig. 7, it consists of an encoder that grad-

ually downsamples the temporal resolution while increasing feature depth using one 1-D convolutional layer and group normalization layers. The bottleneck captures rich contextual information at a compressed scale. The decoder then upsamples the feature maps back to the original resolution using a 1-D transposed convolutional layer, with a group normalization layer and ReLU activations to maintain stability. Skip connections are added between encoder and decoder stages to fuse low-level and high-level features, enhancing reconstruction quality. The final 1-D convolutional layer with a kernel size of one projects the output back to the input channel dimension.

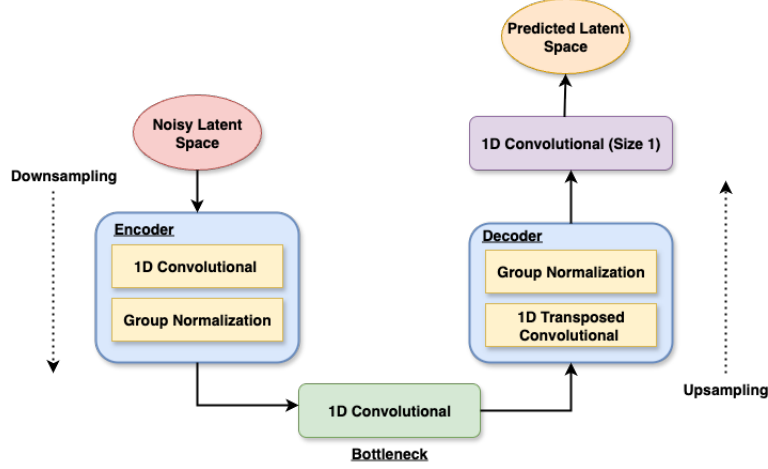


Figure 7: *UNet1D* architecture

In the last step, we convert the latent space data back to the tabular form, using a feedforward sequential neural network-based decoder. It takes the denoised latent vector—typically obtained after the backward diffusion process—and maps it back to the original feature space of the CSV data. The decoder consists of three fully connected (Linear) layers: the first projects the latent vector to 64 dimensions, the second expands it to 128 dimensions, and the final layer outputs a vector of size equal to the original data dimension (output dimension). ReLU activation functions between the layers introduce non-linearity, enabling the model to learn complex mappings. This decoder effectively reverses the compression performed by the encoder and generates realistic tabular data from latent vectors.

5.2. ML Model for MQTT Traffic Classification

In *MQTTSec* framework, we utilize a machine learning model to classify the MQTT traffic and generate the “state” required for the RL model. To

train the model, we use publicly available datasets and synthetic datasets generated using LDM. From the available dataset ⁷, we determine the most important features on which the classification depends. To achieve this, we perform a point-biserial correlation test based on Pearson’s correlation coefficient, which is typically used to measure the correlation between a continuous variable and a binary variable. In this dataset, the binary dependent (target) variable is “target,” whose value can be either “legitimate” or “dos.” Fig. 8 represents the strongly correlated features concerning the target column. We observe that the features like “mqtt.len” and “tcp.len” have a positive correlation with the “target,” indicating that with an increase in the length of the MQTT or TCP message, the chances of occurrence of the DoS attack increase. Since both “mqtt.len” and “tcp.len” are positively correlated, we consider only the first feature in context with this work. On the other hand, the feature column “tcp.time_delta” has a negative correlation with the “target” column values, indicating that with an increase in the time difference between two consecutive sent packets, the chances of the occurrence of a DoS attack decrease. These correlations help in designing the policies for the RL module.

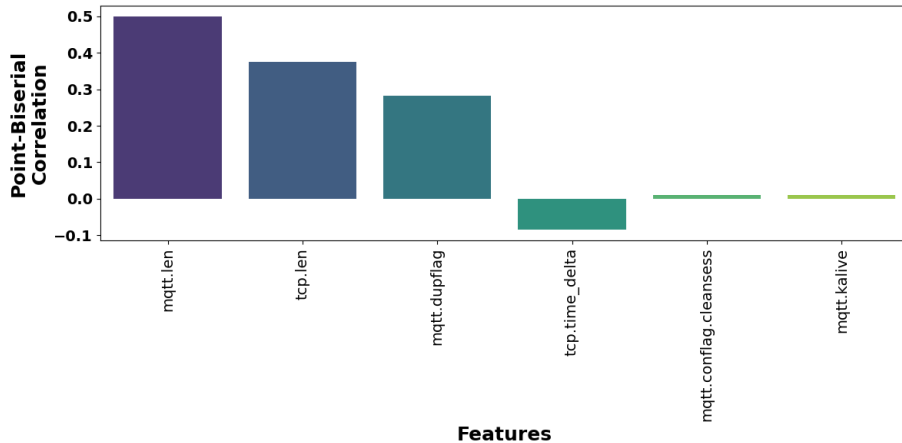


Figure 8: Point Biserial Correlation Test on Available Dataset

For the initial training of the ML model using the available and generated datasets, we consider well-known supervised learning classification algorithms, namely Naive Bayes, Multi Layer Perceptron, XGBoost, Random Forest, Logistic Regression, and Adaboost (refer to Sec. 6.3.1 for perfor-

⁷<https://www.kaggle.com/datasets/cnrieit/mqttset>

mance evaluation). Finally, the optimal classification algorithm is selected for integration with the *MQTTSec* framework. This classification model is retrained at a regular interval to learn the structure of new MQTT traffic and provide accurate state information to the RL agent.

5.3. Reinforcement Learning (RL) Model

This section discusses the working principle of the RL model in the context of the proposed *MQTTSec* framework. The RL model enables the MQTT broker to take episode-level and epoch-level actions based on the policy designed using the type of message.

5.3.1. Episode Level Actions

At the episodic level, the MQTT broker receives the traffic and its RL state as either "attack traffic" or "benign traffic," denoted as 1 and 0, respectively, based on the classification done by the ML model. The following scenarios are possible with three types of messages:

MQTT Connection Establishment: The MQTT clients (publisher/subscriber) need to establish a connection with the broker using the TCP CONNECT message. The broker authenticates these clients and allows them to access network services. Fig. 9 shows the phases of TCP connection establishment between the client and the MQTT broker.

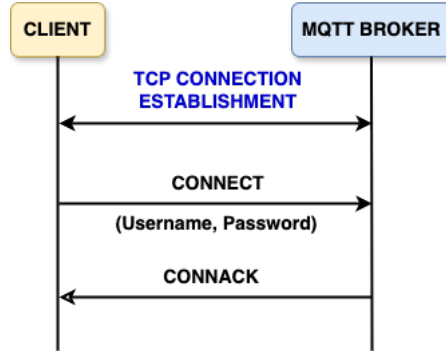


Figure 9: Connection Setup: The client sends a CONNECT message with a small payload containing username and password to the MQTT broker, and the MQTT broker sends a CONNACK message as acknowledgment.

Now, if the client is malicious, it may continue sending a large volume of CONNECT messages with larger payloads at a fast rate. This will be reflected in larger values of the MQTT payload length (L) and a decrease in time delta (t_δ). We define the policies for episode-level actions $\mathbf{A}_1$ based

on the MQTT connection establishment as follows:

$$\mathbf{A}_1 = \begin{cases} \text{accept}, & L \leq \theta_{1min} \ \&\& \ t_\delta \geq \theta_{2max} \\ \text{warn}, & L > \theta_{1min} \ \&\& \ t_\delta < \theta_{2max} \text{ and } L \leq \theta_{1max} \ \&\& \ t_\delta \geq \theta_{2min} \\ \text{decline}, & L > \theta_{1max} \ \&\& \ t_\delta < \theta_{2min} \end{cases} \quad (5)$$

From the available MQTT dataset, we derive the maximum and minimum values of the thresholds θ_1 and θ_2 , as θ_{1min} , θ_{1max} , θ_{2min} , and θ_{2max} .
QoS 1 Message Passing: As depicted in Fig. 2(b), QoS 1 messages are typically exchanged after the establishment of a connection between the clients and the broker. It is similar to MQTT connection establishment, with the only difference being that the message and the acknowledgment are PUBLISH and PUBACK, respectively. The actions taken for this message will be similar to Eqn. 5.

QoS 0 Message Passing: The QoS 0 is the simplest form of message exchange, where the broker does not send any acknowledgement. A DoS-like scenario typically occurs here when the client sends multiple PUBLISH messages within a short duration of time. Therefore, we only need the t_δ to frame the policies, while L is not required. We devise the policies for action $\mathbf{A}_2$ as follows:

$$\mathbf{A}_2 = \begin{cases} \text{accept}, & t_\delta \geq \theta_{2avg} \\ \text{decline}, & t_\delta < \theta_{2avg} \end{cases} \quad (6)$$

The actions defined for this kind of message are two-fold, *accept* and *decline*. The broker accepts the message when the $t_\delta \geq \theta_{2avg}$, and it declines when $t_\delta < \theta_{2avg}$. The value of θ_{2avg} is calculated as the average of the minimum and maximum value of θ_2 , denoted as $\frac{\theta_{2max} + \theta_{2min}}{2}$.

The thresholds θ_1 and θ_2 should be designed such that they are adaptable and adjust to the dynamic network conditions. The threshold θ_1 , related to MQTT payload length L , changes dynamically depending on the available bandwidth and the network throughput (measured in bits per second). Whereas, the threshold θ_2 , related to t_δ , varies with the round-trip time (RTT) (measured in seconds) that is adjusted based on the network congestion. Below we give one possible approach (not limited to) to defining the two thresholds:

$$\theta_1 = \beta \times D_{max} \times \frac{RTT_{max}}{2} \quad (7)$$

$$\theta_2 = \alpha \times \frac{RTT_{max}}{2} \quad (8)$$

Where D_{max} and RTT_{max} are the maximum network throughput and round-trip times in the MQTT-enabled network. In our case, we are only considering one-way message transfer from the client to the broker; therefore, we only take half of the maximum RTT . The parameters α and β can take values from the interval $[0, 1]$, and are set by the MQTT broker to dynamically adjust the thresholds in response to varying network conditions.

5.3.2. Epoch Level Actions

In *MQTTSec*, we perceive a real-time environment where multiple clients can publish or subscribe to the broker simultaneously. There can be multiple reasons that may lead to a DoS-like scenario, which necessitates devising stochastic actions to leave a margin for exploration. The actions taken in the episodes result in rewards that are aggregated over the episodes and are used to calculate the discounted return at the end of n episodes or an epoch. Based on the return and ϵ -greedy approach, the broker determines which action to take against the client. The actions taken are “suspend” from or “continue” in the network using Eqn. 9.

$$\mathbf{A}_E = \begin{cases} Continue, & G_{ti} > \tau_E \\ Suspend, & \text{otherwise} \end{cases} \quad (9)$$

Where G_{ti} refers to the expected discounted return calculated for each client i , and τ_E is the epoch level threshold. If the calculated return for the clients is greater than the threshold, the client can continue operations in the network; otherwise, it is suspended. The expected discounted return calculated is given in Eqn. 10 as:

$$G_{ti} = R_{ti+1} + \gamma R_{ti+2} + \gamma^2 R_{ti+3} + \dots = \sum_{k=0}^n \gamma^k R_{t+k+1} \quad (10)$$

The clients aim to maximize their individual expected return using the reward R and discount factor γ . The G_{ti} for a particular client i refers to the expected return it might receive in the interval $[t + 1, t + k + 1]$, i.e., from the first episode to the k^{th} episode of the next epoch.

If the client is suspended from the network, then the rewards for that particular client are removed from the broker’s memory, and it shall not be able to connect with the same broker for a preconfigured time period Δ .

Fig. 10 provides a schematic representation of the working of the RL model of *MQTTSec* as discussed above.

ϵ -Greedy Approach: For this approach, a reward buffer is used that stores the corresponding state and action value for each action $a \in \Lambda$ taken at each state $s \in S$ over the episodes, and the reward obtained by acting a in a state s . With probability ϵ , the broker explores random actions (irrespective of the return value and the threshold, the client may be allowed in the network or suspended), and with probability $1 - \epsilon$, the broker exploits the best action (if the return is higher than the threshold, then continue; otherwise, suspend the client from the network).

In *MQTTSec* framework, the value of the ϵ is derived from the classification error of the ML model in the final episode of the current epoch. In general, the error of a classification model reflects its inability to correctly classify unseen samples. If action against clients is to be taken by the RL agent based on these misclassified samples, it cannot be done deterministically; rather, it has to be done stochastically. Thus, the agent needs to do ‘exploration’ over the set of possible actions since it can never be certain if the clients have any objective to create a DoS-like scenario. To this end, we vary the value of ϵ in proportion to the error, thereby incorporating stochasticity into the actions taken by the RL agent. Thus, if the error is on the higher side, the likelihood of selecting a random action increases, and vice versa. The epoch-level action is redefined by incorporating ϵ -greedy approach:

$$\mathbf{A_E} = \begin{cases} \text{Return based action,} & \text{with probability } (1-\epsilon) \\ A \text{ random action,} & \text{with probability } \epsilon \end{cases} \quad (11)$$

The value of the ϵ is updated using the classification error (Υ_n) of the n^{th} episode of the latest epoch:

$$\epsilon = \epsilon_r + \frac{\Upsilon_n}{2} \quad (12)$$

Here, ϵ_r is a residual ϵ to prevent it from reaching 0, thus avoiding total exploitation and no exploration. The division by two indicates a steady and moderate increase in the value of ϵ , allowing the model to learn better instead of making drastic jumps in exploration due to large error terms.

6. Results and Discussions

This section presents the results related to traffic generation using LDM, traffic classification using ML models, and the performance of the RL in

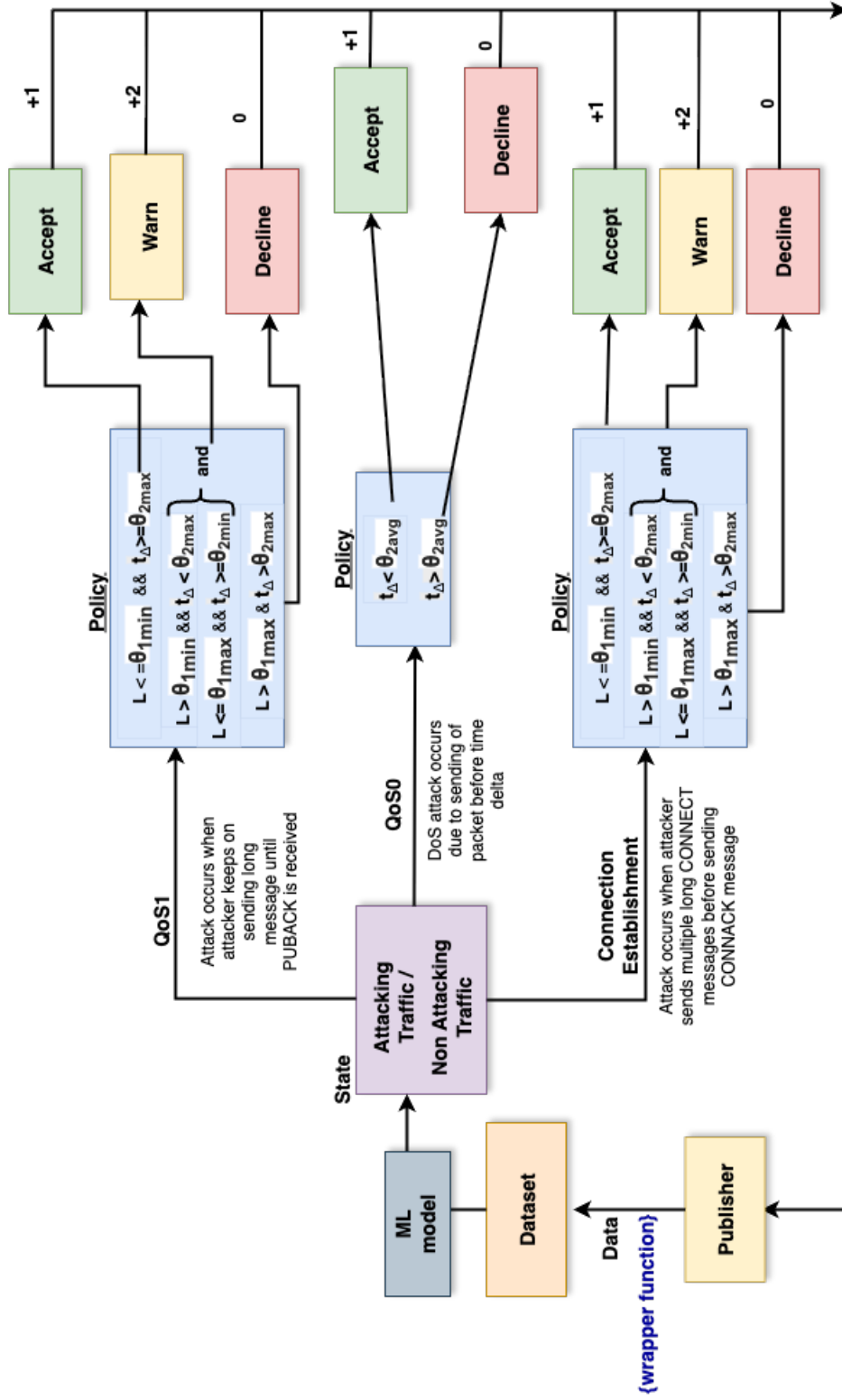


Figure 10: Reinforcement Learning Module in MQTTSec Framework

taking episode-level and epoch-level actions. We implement the *MQTTSec* framework on the MacOS environment Ventura 13.5 with hardware specifications as follows: M2 pro-chip with 12-core CPU, 16 GB RAM, 512 GB SSD storage. We use Python 3.12.0 for building the LDM, ML, and RL models. For LDM, we use the PyTorch 2.7.1 library, and for the ML/RL implementation, we use the scikit-learn 1.6.1 library. The implementation details and codes are available publicly in our repository ⁸. The values of the parameters used in experiments are depicted in Table 2.

Table 2: Parameters and Their Values

Parameters	Values
θ_{1min}	8 Bytes
θ_{1max}	650 Bytes
θ_{2min}	5 ms
θ_{2max}	55 ms
No. of episodes per epoch (n)	10
No. of epochs considered for simulation	75
ϵ_r	0.1
E (No. of epochs after which ML model is retrained)	15
τ_E	18

6.1. Pre-processing of the Input Dataset

We use the dataset available in Kaggle ⁹ as the preliminary dataset to train the ML model. It contains normal MQTT traffic and different types of MQTT DoS attack traffic. From this dataset, we curate a new dataset by selecting only the normal traffic (labeled as 'legitimate') and the traffic labeled as 'DoS' (refer to Fig. 11 (a)). For preprocessing, we remove the *NaN* values and the empty cells and prepare the dataset for the initial epoch's ML model training. We train the LDM using the curated dataset as its input and generate a new dataset as its output (refer to Fig. 11 (b)). The synthetic dataset is in CSV format and is used for training and testing the ML model in future epochs.

6.2. Dataset Generation using Latent Diffusion Model

Fig. 12 provides a heatmap comparison of a subset of the original and the generated data. The X -axis represents the 26 features of the publicly

⁸<https://github.com/debasmitadey9/MQTTSec.git>

⁹<https://www.kaggle.com/datasets/cnrieit/mqttset>

tcp.time_delta	tcp.len	mqtt.conack.flags	mqtt.conack.flags.reserved	mqtt.conack.flags.sp	mqtt.conack.val	mqtt.conflag.cleansess
0.998867	10	0	0.0	0.0	0.0	0.0
6.7E-05	1460	0	0.0	0.0	0.0	0.0
5.8E-05	1460	0	0.0	0.0	0.0	0.0
0.000227	10	0	0.0	0.0	0.0	0.0
0.000236	16	0	0.0	0.0	0.0	0.0

(a)

tcp.time_delta	tcp.len	mqtt.conack.flags.reserved	mqtt.conack.flags.sp	mqtt.conack.val	mqtt.conflag.cleansess
0.000000	73.206085	0.000000	0.000000	0.000000	0.000599
0.000000	75.424896	0.000262	0.000290	0.000231	0.000234
0.000000	641.334839	0.000007	0.000000	0.000000	0.000532
0.062301	162.852127	0.000000	0.000797	0.002074	0.000000
0.000000	81.236938	0.000224	0.000271	0.000241	0.000167

(b)

Figure 11: Dataset: (a) Original (b) Generated

available dataset, and the Y-axis represents a sample of 100 rows (or packets) of the dataset. As evident in Fig. 12(a), the dataset has only two significant features (nos 1 and 15) representing “time delta” and “mqtt_length.” The values of the rest of the features in the original dataset are 0, thus resulting in the depicted heatmap. Fig. 12(b) shows that these two features are reasonably replicated in the generated samples, indicating that the most important features on which the decision of the MQTT broker is dependent are available in the generated dataset as well.

This observation can be further substantiated by the Kullback-Leibler (KL) divergence study that quantifies the difference between the probability distribution of the normalized feature values of “mqtt_len” and “time_delta” in both the original dataset and the generated datasets. Fig. 13 shows the KL-divergence result that illustrates a high degree of overlap between the probability distribution curves.

In [30], the authors establish that diffusion models perform better in network packet generation compared to GAN, and the LDM converts the high-dimensional data into a low-dimensional latent space before starting the process of diffusion. This ensures achieving convergence after a minimum number of epochs. Fig. 14 represents the loss calculated using Eqn. 3 for the LDM for generating the synthetic data. The loss attains convergence from the 20th episode onwards.

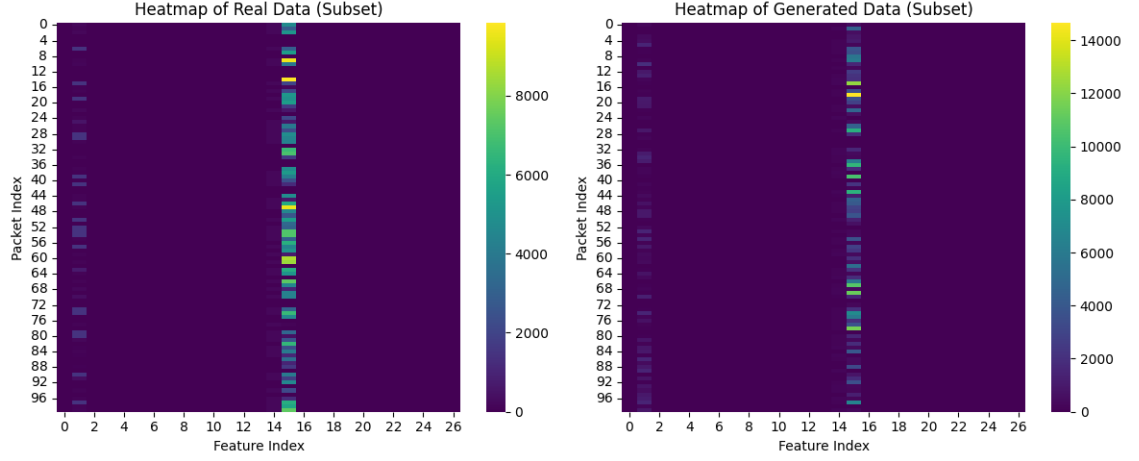


Figure 12: Heatmap of the Real and Generated Data

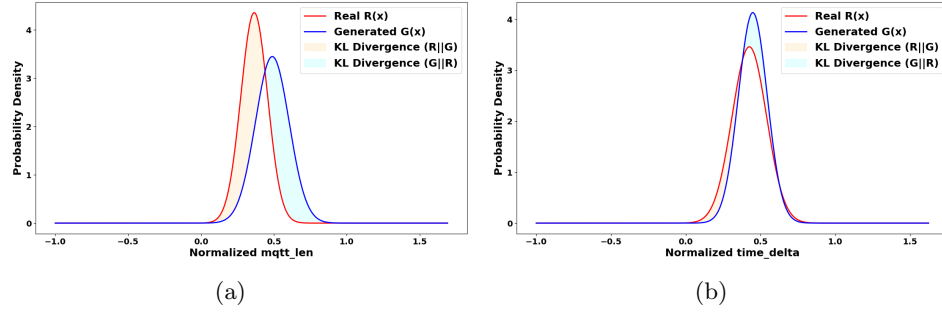


Figure 13: (a) KL Divergence of Normalized mqtt_len feature from Real and Generated Datasets (b) KL Divergence of Normalized time_delta feature from Real and Generated Datasets

6.3. Classification of MQTT Traffic using ML Model

As discussed in Sec. 4, the RL model in the *MQTTSec* framework uses the classification results from the ML model as its states. The MQTT traffic can be classified into two classes: “attack” and “benign.” Since we simulate a real-world MQTT-enabled network, the continuous flow of MQTT traffic generates the data samples required for training, testing, and retraining the ML model. We use 80% of the samples for training purposes and the rest to test the model’s performance in terms of classification accuracy. A wrapper function is used to prepare the samples for the ML model by converting the MQTT packets from the publisher or subscriber into raw data and saving them in CSV files.

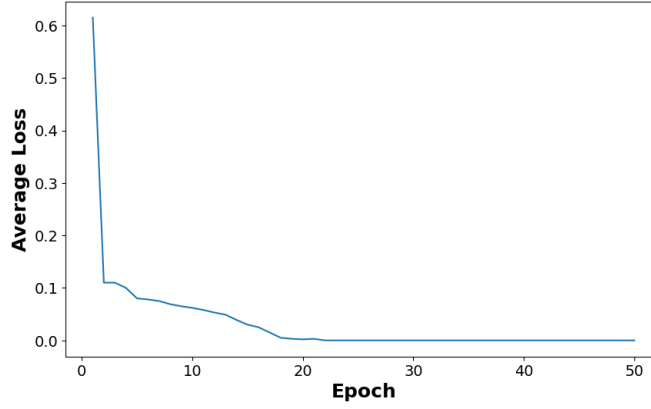


Figure 14: Average Loss of the Latent Diffusion Model

6.3.1. Selecting the Optimal ML Model

To find the optimal ML model, we test various models and measure their accuracy. Consequently, we select the model that yields the best test accuracy.

Table 3: Accuracy of Various ML Models at the Initial Stage

Classification Models	Accuracy (%)
Naive Bayes	87.49
Multi Layer Perceptron	87.5
XGBoost	95.91
Random Forest	95.92
Logistic Regression	87.5
Adaboost	95.47

From Table 3, we find *random forest* to be the optimal model that gives the best classification accuracy. We select this ML model for the initial training and retraining in subsequent epochs.

6.3.2. Retraining the Selected ML Model

We retrain the ML model after a time period of E epochs to take into account the dynamic nature of the network, where different types of MQTT messages are shared between publishers and subscribers via the MQTT broker. This traffic is largely unseen by the ML model and there is a need to incorporate it into the training set and carry out retraining at periodic intervals (after every E epochs) to prevent performance degradation. The effect of retraining and its relation with the ϵ value is given in Sec. 5.3.2.

6.4. Results for Reinforcement Learning

This subsection presents results related to policy and actions taken by the RL model of *MQTTSec*.

6.4.1. Effects of Parameters on RL Actions

We study the effects of parameters “mqtt_len” (message length) and “time_delta” (time interval between successive messages) on the RL actions.

Variation of Message Length: Fig. 15 depicts the effect of the variation of the message length on the number of actions taken in case of the QoS 1 and “Connection Establishment” message types. We calculate the average number of actions over 10 episodes for a particular client. The threshold MQTT message lengths are set at 8 Bytes (θ_{1min}) and 650 Bytes (θ_{1max}) respectively.

Fig. 15 (a) showcases a count of each type of action taken throughout 10 episodes, while processing the “Connection Establishment” messages with a variation of the message length from 8 to 650 Bytes. Meanwhile, Fig. 15 (b) showcases the same, but while handling the QoS 1 messages. It can be observed from the results, when the message length is shorter, the number of “accept” actions taken by the RL agent is more than the number of “warn” and “decline” actions. On the other hand, when the message length increases, the number of “decline” actions is considerably more than the number of “accept” and “warn” actions taken. In both cases, a longer message length indicates that the payload can exhaust the MQTT broker’s memory, leading to a DoS-like situation. As can be seen, for message lengths of 460 and 660 Bytes, the number of “decline” actions is more than the number of “accept” actions. Therefore, with *MQTTSec*, the length of the message can be used as a useful indicator to protect against potential DoS attacks.

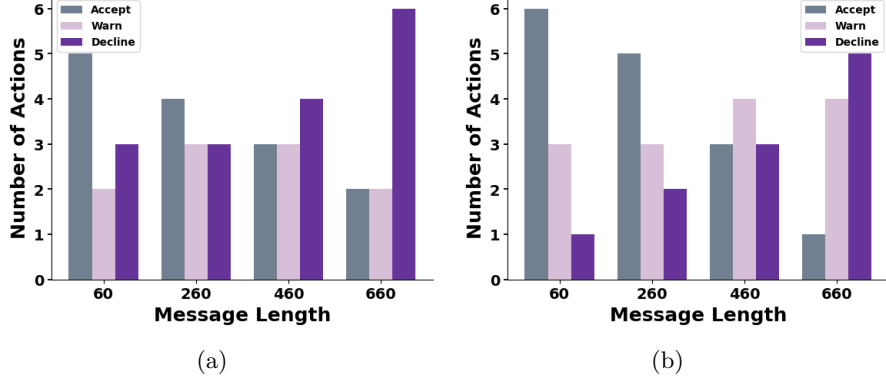


Figure 15: Actions Taken under various window sizes for: (a) Connect Message (b) QoS1 Message

Variation of Time delta: In a similar way, we compute results related to time delta values on the three message types: (i) connection establishment message, (ii) QoS 0, and (iii) QoS 1. We maintain the threshold θ_{2min} at 5 ms, θ_{2max} at 55 ms, and θ_{2avg} at 30 ms for all the message types. Fig. 16 represents the number of each type of action taken for time delta values varying from 15 ms to 60 ms. A lower time delta value indicates a lesser time gap between the transmission of two messages; thus, indicating that the sender can send messages at a higher rate without waiting for a longer time. On the other hand, a higher time delta value indicates a larger time gap between the transmission of two messages; thus, indicating that the sender needs to wait for a considerable amount of time before sending out the next message.

For the QoS 0 message (see Fig. 16 (b)), we have two types of actions, namely “accept” and “decline”, and it can be observed that for lower time delta value, when the sender need not wait much longer before sending another message, the number of “decline” action is relatively higher than the other two actions. The reason for this is that a higher sending rate overwhelms the MQTT broker, leading to a DoS situation, hence the decline of packets. In contrast, we observe that by increasing the value of the time delta, where the client takes more time to send the next message, the number of “decline” actions is relatively less than that of “accept” actions. Finally, in the QoS 1 and connection establishment messages (see Fig. 16 (a) and (c) respectively), there is an additional action, namely the “warn”, where the broker alerts the sender on anticipation that the latter’s activity might lead to a DoS-like situation in the near future.

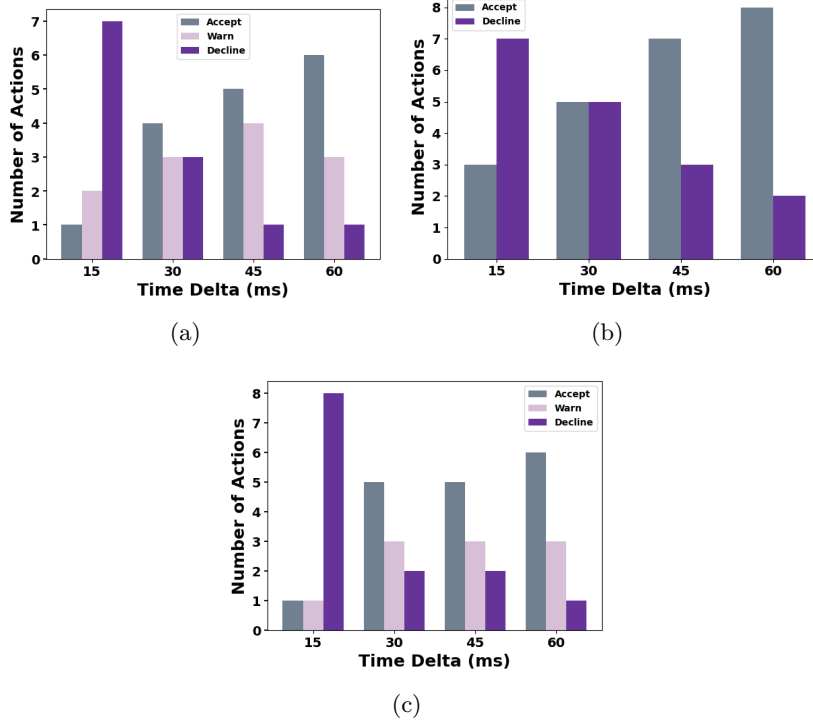


Figure 16: Actions in various Time Delta values for message types: (a) Connect (b) QoS 0 (c) QoS 1

6.4.2. Return based on Traffic Types

We consider three types of traffic conditions to test the performance of the ML and RL models, where the publishers transfer “70% benign”, “80% benign”, and “90% benign” traffic to the broker. For each type, we calculate the return value over 100 epochs. Fig. 17 shows the returns obtained in the three settings. It can be seen that for the “70% benign” data, the median return is lowest compared to the other two, indicating the smallest return values. The highest median value and subsequently the highest return is for the “90% benign” traffic. Moreover, the range of return values for the “70% benign” traffic is relatively wider, indicating higher variance and a more volatile condition due to a higher possibility of a DoS attack. In contrast, for the 80% and 90% benign traffic, the spread is more tightly grouped, indicating a more stable condition in the absence of attacks.

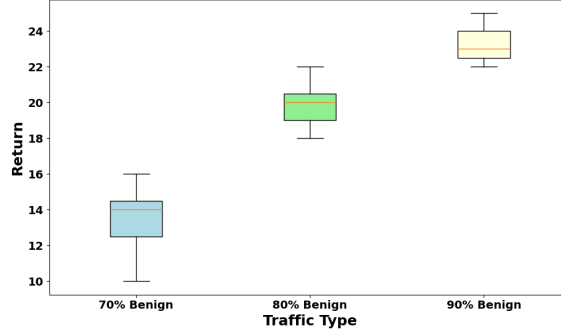


Figure 17: Return based on Traffic

6.4.3. ϵ -Greedy Approach in Decision Making

As discussed in Sec. 5.3.2, the optimal actions taken by the RL agent after an epoch are: (i) to continue with clients in the network if their discounted return is more than the epoch level threshold (τ_E), and (ii) to suspend the clients if their discounted return is less than the threshold. Before making any decision on actions, an ϵ -greedy approach is adopted to impart stochasticity. This is largely due to the uncertainty with clients' behaviors that may often be driven by network dynamics. In Fig. 18, we alter the ϵ values and observe optimal actions taken over 100 epochs.

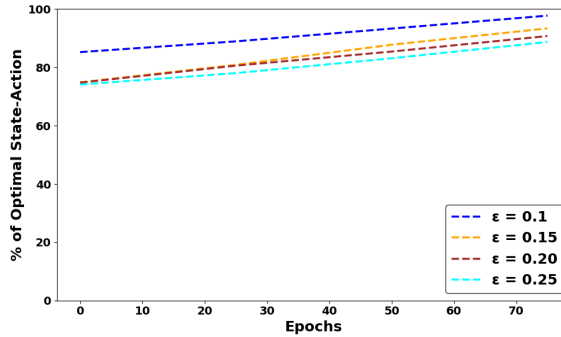


Figure 18: Effect of ϵ values on Optimal Action

If the ϵ value is higher, the probability of “exploration” increases, which governs the agent to take a different action than the optimal one. On the other hand, with a lower ϵ value, the probability of “exploitation” increases, leading to taking the optimal action determined by the discounted return value. However, for all values of ϵ , the agent eventually learns to take the optimal action consistently over several epochs.

Choosing the optimal value for ϵ is challenging as both network condi-

Table 4: Epoch Level Actions with State

	Error (%)	ϵ	Avg. Optimal Action (%)	Avg. Random Action (%)
<i>Epoch 16</i>	5.58	0.127	78	22
<i>Epoch 31</i>	9.19	0.145	81	19
<i>Epoch 46</i>	7.28	0.136	83	17
<i>Epoch 61</i>	5.44	0.127	86	14
<i>Epoch 76</i>	6.89	0.134	88	12
<i>Epoch 91</i>	6.68	0.134	91	9

tions and clients’ behavior have to be taken into account. As discussed in Sec. 5.3.2, at a given epoch, we use the MQTT traffic classification error generated after periodically retraining the ML model to be proportional to the ϵ value. For experimental purposes, we retrain the model after every $E = 15$ epochs, using the synthetic data generated by the LDM as the training samples.

Table 4 gives the errors of the retrained ML model and the corresponding ϵ values calculated using Eq. 11 after every 15 epochs. It also presents the percentages of optimal and random actions taken by the RL agent averaged out over 15 epochs. During initial epochs, the average percentage of the optimal action is 78%, while that for random action is 22%. However, with passing epochs, the RL model learns to take the optimal action, that is, to continue with the client when the return is high, and suspend the latter from the network when the return is low. This is reflected by the gradual increase in the percentage of optimal action and drop in the random action. Thus, the RL agent is learning with time to take the optimal action even with larger ϵ values, which indicates less “exploitation” and more “exploration”.

7. Conclusion

MQTT is a publish-subscribe application layer protocol for IoT networks, generally used for message exchanges. However, there have been occurrences of DoS attacks in multiple scenarios due to the basic CONNECT flooding, heavy flooding, and fast flooding during MQTT message exchanges. The state-of-the-art defense mechanism is limited to the detection of DoS attacks

using ML classifiers trained on publicly available MQTT traffic datasets. However, preventing DoS attacks on-the-fly is largely unavailable in the state-of-the-art literature. Taking preventive measures against potential DoS attacks is challenging, as in a real-time scenario, honest clients may behave abnormally (sending longer or frequent packets) due to faulty network conditions. In *MQTTSec*, a DoS attack is prevented by enabling the MQTT broker to analyze clients' behavior and issue a warning signal before suspending them from the network. For validation, we simulate an MQTT network consisting of one broker and two publishers. The broker takes action decisions against publishers powered by ML and RL models, which classify the incoming messages as attack or benign. To address the dearth of MQTT traffic data, we use a latent diffusion model, generate synthetic traffic, and use it for periodic retraining of the ML model. We show results regarding the synthetic generation of datasets, ML classification of incoming packets, and RL-driven actions. As future work, we will attempt to enhance the capability of the RL agent to prevent other variants of DoS and make the *MQTT* framework more scalable to support a larger number of publishers and subscribers.

References

- [1] V D Panduranga Sai Guptha, Satish Anamalamudi, Reddy Priya Madupuri, and Sruthi Sivarajan. Interoperability between opc-ua and mqtt in hybrid industrial iot(iiot) environments. In *2025 17th International Conference on COMMunication Systems and NETWORKS (COM-SNETS)*, pages 1189–1194, 2025.
- [2] Erkin Kirdan, Patricia Horvath, and Marc-Oliver Pahl. Work-in-progress: Slow denial of service attack on mqtt-based iot. In *2023 IEEE International Black Sea Conference on Communications and Networking (BlackSeaCom)*, pages 426–431, 2023.
- [3] Haripriya Ap and Kulothungan K. Secure-mqtt: an efficient fuzzy logic-based approach to detect dos attack in mqtt protocol for internet of things. *EURASIP Journal on Wireless Communications and Networking*, 2019(1):90, 2019.
- [4] Naeem Firdous Syed, Zubair Baig, Ahmed Ibrahim, and Craig Valli. Denial of service attack detection through machine learning for the iot. *Journal of Information and Telecommunication*, 4(4):482–503, 2020.

- [5] Munmun Swain, Nikhil Tripathi, and Kamalakanta Sethi. Identifying communication sequence anomalies to detect dos attacks against mqtt. *Computers & Security*, page 104526, 2025.
- [6] Ali Ghannadrad. Machine learning-based dos attacks detection for mqtt sensor networks. 2020.
- [7] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models, 2022.
- [8] Umberto Morelli, Ivan Vaccari, Silvio Ranise, and Enrico Cambiaso. Dos attacks in available mqtt implementations: investigating the impact on brokers and devices, and supported anti-dos protections. In *Proceedings of the 16th International Conference on Availability, Reliability and Security*, pages 1–9, 2021.
- [9] Ivan Vaccari, Maurizio Aiello, and Enrico Cambiaso. Slowite, a novel denial of service attack affecting mqtt. *Sensors*, 20(10):2932, 2020.
- [10] Fu Chen, Yujia Huo, Jianming Zhu, and Dan Fan. A review on the study on mqtt security challenge. In *2020 IEEE International Conference on Smart Cloud (SmartCloud)*, pages 128–133. IEEE, 2020.
- [11] Giuseppe Potrino, Floriano De Rango, and Amilcare Francesco Santamaria. Modeling and evaluation of a new iot security system for mitigating dos attacks to the mqtt broker. In *2019 IEEE Wireless Communications and Networking Conference (WCNC)*, pages 1–6. IEEE, 2019.
- [12] Hayette Zeghida, Mehdi Boulaiche, and Ramdane Chikh. Detection of dos attacks in mqtt environment. In *International Conference on Intelligent Systems and Pattern Recognition*, pages 129–140. Springer, 2023.
- [13] Dmitrii Dikii, Sergey Arustamov, and Aleksey Grishentsev. Dos attacks detection in mqtt networks. *Indonesian Journal of Electrical Engineering and Computer Science*, 21(1):601–608, 2021.
- [14] OASIS Standard. Mqtt version 3.1.1 specification. <http://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html>, 2014.

- [15] Andrew Banks and Rahul Gupta. Mqtt version 5.0 specification. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>, 2019.
- [16] Claire Rowland, Elizabeth Goodman, Martin Charlier, Ann Light, and Alfred Lui. *Designing Connected Products: UX for the Consumer Internet of Things*. O’Reilly Media, Inc., 2020.
- [17] Adriel Cheng. Pac-gan: Packet generation of network traffic using generative adversarial networks. In *2019 IEEE 10th Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON)*, pages 0728–0734. IEEE, 2019.
- [18] Debasmita Dey and Nirnay Ghosh. iquic: An intelligent framework for defending quic connection id-based dos attack using advantage actor-critic rl. *Computers & Security*, page 104463, 2025.
- [19] Hoang Thanh-Tung and Truyen Tran. Catastrophic forgetting and mode collapse in gans. In *2020 international joint conference on neural networks (ijcnn)*, pages 1–10. IEEE, 2020.
- [20] Danial Khosh Kholgh and Panos Kostakos. Pac-gpt: A novel approach to generating synthetic network traffic with gpt-3. *IEEE Access*, 11:114936–114951, 2023.
- [21] Tuo An, Yunjiao Zhou, Han Zou, and Jianfei Yang. Iot-llm: Enhancing real-world iot task reasoning with large language models. *arXiv preprint arXiv:2410.02429*, 2024.
- [22] Soumyadeep Hore, Jalal Ghadermazi, Diwas Paudel, Ankit Shah, Tapas Das, and Nathaniel Bastian. Deep packgen: A deep reinforcement learning framework for adversarial network packet generation. *ACM Transactions on Privacy and Security*, 28(2):1–33, 2025.
- [23] Shiyuan Zhang, Haoye Chai, Yong Li, Baohua Qiu, Lei Yue, and Raoyuan Pan. Packetdiff: A flow guided diffusion model for network packet trace generation. *IEEE Internet of Things Journal*, 2025.
- [24] Abdullah Bin Jasni, Akiko Manada, and Kohei Watabe. Diffupac: Contextual mimicry in adversarial packets generation via diffusion model. *Advances in Neural Information Processing Systems*, 37:133846–133878, 2024.

- [25] Thomas B. Farver. Chapter 1 - concepts of normality in clinical biochemistry. In J. Jerry Kaneko, John W. Harvey, and Michael L. Bruss, editors, *Clinical Biochemistry of Domestic Animals (Sixth Edition)*, pages 1–25. Academic Press, San Diego, sixth edition edition, 2008.
- [26] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.
- [27] Alaa Alatrham, Leslie F. Sikos, Mike Johnstone, Patryk Szewczyk, and James Jin Kang. Dos/ddos-mqtt-iot: A dataset for evaluating intrusions in iot networks using the mqtt protocol. *Computer Networks*, 231:109809, 2023.
- [28] Mustafa Al-Fayoumi and Qasem Abu Al-Haija. Capturing low-rate ddos attack based on mqtt protocol in software defined-iot environment. *Array*, 19:100316, 2023.
- [29] Daniel Stoller, Mi Tian, Sebastian Ewert, and Simon Dixon. Seq-u-net: A one-dimensional causal u-net for efficient sequence modelling, 2019.
- [30] Jussi Leinonen, Ulrich Hamann, Daniele Nerini, Urs Germann, and Gabriele Franch. Latent diffusion models for generative precipitation nowcasting with accurate uncertainty quantification. *arXiv preprint arXiv:2304.12891*, 2023.